THE COMPLETE

GITHUB STARTER PACK

Building a Recruiter-Ready Engineering Profile

Profile READMEs • Project Documentation • Git & GitHub Mastery
Conventional Commits • Open Source Contribution • Licensing
Issue & PR Templates • Placement-Ready Best Practices

EDITION 2026

For Engineering Students & Early-Career Software Developers

Created by

Ansh Banga

linkedin.com/in/ansh-banga-775275330

Table of Contents

PART ONE

GitHub Profile README Template

The first file a recruiter opens - treat it like the homepage of your personal brand.

Your GitHub profile README (the special repository named exactly after your username) is the single most under-used asset in a student's job search. It sits above your pinned repositories and is the first thing a recruiter, hiring manager, or open-source maintainer sees when they click your avatar.

This chapter gives you a complete, production-ready template, section by section, along with the reasoning behind every design choice.

> **RECRUITER INSIGHT**
> Recruiters spend an average of 15-30 seconds scanning a GitHub profile before deciding whether to dig deeper. Your README has one job: convert that glance into a click on your pinned projects.

1.1 Anatomy of a High-Impact Profile README

A profile that reads well end-to-end follows a predictable rhythm: hook, credibility, proof, and a call to action. The eight blocks below map directly onto that rhythm.

Block	Purpose	Ideal Length
Professional Introduction	Hook + one-line positioning statement	3-5 lines
Skills	Scannable proof of technical range	1 grid / table
Tech Stack Badges	Visual credibility, ATS keyword coverage	2-3 rows of badges
GitHub Stats	Quantified activity and consistency	1-2 stat cards
Contribution Graph	Visual proof of consistency over time	1 embedded image
Social Links	Frictionless path to LinkedIn/Portfolio	1 badge row
Portfolio	Deep-dive proof of shipped work	3-5 project links
Contact	Clear, low-friction call to action	1-2 lines

1.2 Ready-to-Use Template

Create a repository with the exact same name as your GitHub username, add a README.md, and paste the structure below. Replace every bracketed placeholder - do not leave sample text in the final version.

Professional Introduction

```
<h1 align="center">Hi, I'm [Your Full Name] 👋</h1>
<h3 align="center">[Your Title] - [Your One-Line Positioning Statement]</h3>
```

```
<p align="center">
  I build [type of product, e.g. "scalable web applications"] using [core stack].
  Currently exploring [current learning focus, e.g. "distributed systems"] and
  contributing to open source in my free time.
</p>

- 🔭 Currently working on: **[Project Name]**
- 🌱 Currently learning: **[Technology/Concept]**
- 👯 Looking to collaborate on: **[type of project]**
- 💬 Ask me about: **[your strongest domain]**
- ⚡ Fun fact: **[one memorable, human detail]**
```

> **PRO TIP**
> Keep the positioning statement to one sentence and make it role-specific - "Backend Engineer focused on distributed systems" reads far stronger than "Passionate coder and tech enthusiast."

Skills

List skills grouped by category rather than as one long flat list - grouped skills are easier to scan and easier for an ATS parser to map to job requirements.

```
## 🧠 Skills

**Languages:** Python, JavaScript, TypeScript, Java, C++
**Frontend:** React, Next.js, Tailwind CSS, Redux
**Backend:** Node.js, Express, Django, Flask, REST APIs
**Databases:** PostgreSQL, MongoDB, MySQL, Redis
**DevOps & Cloud:** Docker, Git, GitHub Actions, AWS, Vercel
**Tools:** VS Code, Postman, Figma, Jira
```

Tech Stack Badges

Badges served through shields.io render consistently across GitHub, GitLab, and portfolio embeds, and are indexed as plain-text alt attributes, which quietly helps ATS keyword matching when recruiters review the raw README source.

```
## 🛠️ Tech Stack

![Python](https://img.shields.io/badge/Python-3776AB?style=for-the-
badge&logo=python&logoColor=white)
![JavaScript](https://img.shields.io/badge/JavaScript-F7DF1E?style=for-the-
badge&logo=javascript&logoColor=black)
![React](https://img.shields.io/badge/React-61DAFB?style=for-the-
badge&logo=react&logoColor=black)
![Node.js](https://img.shields.io/badge/Node.js-339933?style=for-the-
badge&logo=node.js&logoColor=white)
![Flask](https://img.shields.io/badge/Flask-000000?style=for-the-
badge&logo=flask&logoColor=white)
```

```
![MongoDB](https://img.shields.io/badge/MongoDB-47A248?style=for-the-
badge&logo=mongodb&logoColor=white)
![Docker](https://img.shields.io/badge/Docker-2496ED?style=for-the-
badge&logo=docker&logoColor=white)
![Git](https://img.shields.io/badge/Git-F05032?style=for-the-
badge&logo=git&logoColor=white)
```

GitHub Stats

The stats widgets below are community-maintained image generators - they render live SVGs pulled directly from the GitHub API, so they always stay current.

```
## 📊 GitHub Stats

<p align="center">
  <img src="https://github-readme-
stats.vercel.app/api?username=YOUR_USERNAME&show_icons=true&theme=default&count_private=true
" height="165"/>
  <img src="https://github-readme-stats.vercel.app/api/top-
langs/?username=YOUR_USERNAME&layout=compact&theme=default" height="165"/>
</p>

<p align="center">
  <img src="https://github-readme-streak-
stats.herokuapp.com/?user=YOUR_USERNAME&theme=default" />
</p>
```

Contribution Graph

A 3D or animated contribution graph is optional polish, but the default GitHub graph already tells the real story: consistency. Pair it with a snake-animation workflow for a distinctive visual signature.

```
## 📈 Contribution Graph

![Contribution Graph](https://github-readme-activity-
graph.vercel.app/graph?username=YOUR_USERNAME&theme=react-dark)

<!-- Optional: animated snake graph generated via a scheduled GitHub Action -->
![Snake
animation](https://raw.githubusercontent.com/YOUR_USERNAME/YOUR_USERNAME/output/github-
contribution-grid-snake.svg)
```

Social Links

```
## 🌐 Connect With Me

<p align="left">
```

```
<a href="https://linkedin.com/in/YOUR_LINKEDIN" target="blank"><img
src="https://img.shields.io/badge/LinkedIn-0A66C2?style=for-the-
badge&logo=linkedin&logoColor=white"/></a>
<a href="https://YOUR_PORTFOLIO_URL" target="blank"><img
src="https://img.shields.io/badge/Portfolio-000000?style=for-the-
badge&logo=vercel&logoColor=white"/></a>
<a href="https://twitter.com/YOUR_HANDLE" target="blank"><img
src="https://img.shields.io/badge/Twitter-1DA1F2?style=for-the-
badge&logo=twitter&logoColor=white"/></a>
<a href="mailto:YOUR_EMAIL" target="blank"><img
src="https://img.shields.io/badge/Email-D14836?style=for-the-
badge&logo=gmail&logoColor=white"/></a>
</p>
```

Portfolio

```
## 🗀 Featured Projects

### [Project One Name](https://github.com/YOUR_USERNAME/project-one)
One-line outcome-focused description - what it does and the measurable impact.
`React` `Node.js` `MongoDB` `AWS`

### [Project Two Name](https://github.com/YOUR_USERNAME/project-two)
One-line outcome-focused description - what it does and the measurable impact.
`Python` `Flask` `PostgreSQL`

### [Project Three Name](https://github.com/YOUR_USERNAME/project-three)
One-line outcome-focused description - what it does and the measurable impact.
`Android` `Kotlin` `Firebase`
```

Contact

```
## 📬 Get in Touch

I'm open to internships, entry-level roles, and collaborative open-source work.
Reach out at **your.email@example.com** or connect on LinkedIn - I usually reply
within 24 hours.

---
<p align="center"><i>⭐ If any of these projects are useful to you, consider
starring the repo!</i></p>
```

⚠ COMMON MISTAKE

Do not paste every badge shields.io offers. A profile with 40 badges reads as noise, not skill. Cap Tech Stack badges at 10-12 and only include tools you can defend in an interview.

BEST PRACTICE

> Regenerate your pinned repositories every semester. Recruiters weight recency - a profile that visibly reflects your most recent, most relevant work outperforms one frozen from first year.

PART TWO

Project README Templates

Eight fully worked README templates covering the project types most commonly built by students and early-career engineers.

Every repository you publish should read like a mini case study, not a code dump.

Each template below follows the same twelve-part skeleton - Overview, Features, Tech Stack, Architecture, Installation, Usage, Folder Structure, Screenshots, Future Improvements, License, and Author - so recruiters can navigate any of your repositories using the same mental model.

BEST PRACTICE

Consistency across repositories compounds. A recruiter who finds your README structure predictable will read your third and fourth repository, not just your first.

RECRUITER INSIGHT

Technical recruiters often skim the Installation and Usage sections first to judge how quickly they could run your project. Keep both copy-paste ready with zero unexplained steps.

AI / ML Project README

For classification, regression, computer-vision, or NLP model repositories.

Sample Repository: “Crop Disease Detection using CNN”

Project Overview

An end-to-end deep learning pipeline that classifies plant leaf images into 12 disease categories, packaged as a Flask inference API and a lightweight web demo for field use by agricultural extension workers.

Features

- Custom CNN + transfer learning (ResNet50) with 96.4% validation accuracy
- Data augmentation pipeline (rotation, flip, color jitter) to counter class imbalance
- REST inference endpoint returning class probabilities and Grad-CAM heatmaps
- Experiment tracking with MLflow and reproducible training via config files
- Dockerized inference service for one-command deployment

Tech Stack

Python 3.11 • PyTorch • OpenCV • Flask • MLflow • Docker • NumPy / Pandas

Architecture

- Data Layer - raw images -> preprocessing -> augmented tensors (data/, src/preprocessing.py)
- Model Layer - CNN architecture, training loop, checkpointing (src/model.py, src/train.py)
- Serving Layer - Flask app loads the best checkpoint and exposes /predict (app.py)
- Client - minimal HTML/JS upload form consuming the inference API (templates/)

Installation

```
git clone https://github.com/YOUR_USERNAME/crop-disease-detection.git
cd crop-disease-detection
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Usage

```
# Train the model
python src/train.py --config configs/resnet50.yaml

# Run inference API
python app.py
# Visit http://127.0.0.1:5000
```

Folder Structure

```
crop-disease-detection/
```

```
├── data/               # raw & processed datasets (gitignored)
├── notebooks/          # EDA and experiment notebooks
├── src/
│   ├── preprocessing.py
│   ├── model.py
│   ├── train.py
│   └── evaluate.py
├── configs/            # YAML training configs
├── templates/          # web demo front-end
├── app.py              # Flask inference server
├── requirements.txt
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Upload Interface | Prediction Output | Grad-CAM Heatmap |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Expand dataset to 30+ disease classes across 5 crop types
- Quantize model with ONNX for on-device / edge inference
- Add multilingual voice output for low-literacy field users

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Data Science Project README

For EDA, statistical analysis, and dashboard-driven insight projects.

Sample Repository: "Retail Sales Forecasting & Insights Dashboard"

Project Overview

A statistical analysis and forecasting project that cleans three years of retail transaction data, uncovers seasonal demand patterns, and predicts next-quarter sales using SARIMA and XGBoost, surfaced through an interactive Streamlit dashboard.

Features

- Automated data-cleaning pipeline handling missing values, outliers, and currency normalization
- Exploratory analysis notebooks with 20+ annotated visualizations
- Two forecasting models (SARIMA baseline, XGBoost with lag features) with MAPE comparison
- Interactive Streamlit dashboard for filtering by region, category, and time window
- Automated PDF report generation for stakeholder distribution

Tech Stack

Python 3.11 • Pandas • NumPy • Scikit-learn • XGBoost • Statsmodels • Streamlit • Plotly

Architecture

- Ingestion - raw CSV/Excel exports loaded and validated (src/ingest.py)
- Transformation - cleaning, feature engineering, aggregation (src/transform.py)
- Modeling - SARIMA & XGBoost training and evaluation (src/models/)
- Presentation - Streamlit app rendering charts and forecasts (dashboard/app.py)

Installation

```
git clone https://github.com/YOUR_USERNAME/retail-sales-forecasting.git
cd retail-sales-forecasting
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Usage

```
# Run the full analysis pipeline
python src/run_pipeline.py

# Launch the dashboard
streamlit run dashboard/app.py
```

Folder Structure

```
retail-sales-forecasting/
├── data/
│   ├── raw/
│   └── processed/
├── notebooks/           # EDA & modeling notebooks
├── src/
│   ├── ingest.py
│   ├── transform.py
│   └── models/
├── dashboard/
│   └── app.py           # Streamlit entry point
├── reports/             # generated PDF/HTML reports
├── requirements.txt
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Dashboard Overview | Forecast Chart | Report Export |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Add anomaly detection for fraud-pattern flagging
- Integrate live data source via scheduled ETL (Airflow)
- Deploy dashboard to Streamlit Community Cloud with auth

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Flask Project README

For REST APIs and server-rendered web applications built on Flask.

Sample Repository: "TaskFlow - Team Task Management API"

Project Overview

A production-style Flask REST API for team task management, featuring JWT authentication, role-based access control, and PostgreSQL persistence, built to demonstrate clean layered architecture.

Features

- JWT-based authentication with refresh-token rotation
- Role-based access control (Admin, Manager, Member)
- Full CRUD for projects, tasks, and comments with pagination
- Input validation via Marshmallow schemas and centralized error handling
- Automated test suite (Pytest) with 90%+ coverage and CI on GitHub Actions

Tech Stack

Python 3.11 • Flask • Flask-RESTful • SQLAlchemy • PostgreSQL • Marshmallow • JWT • Pytest • Docker

Architecture

- Presentation - Blueprints define route groups per resource (app/routes/)
- Service - business logic isolated from request/response handling (app/services/)
- Data - SQLAlchemy models and migrations (app/models/, migrations/)
- Config - environment-based configuration classes (config.py)

Installation

```
git clone https://github.com/YOUR_USERNAME/taskflow-api.git
cd taskflow-api
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
flask db upgrade
```

Usage

```
# Set environment variables
export FLASK_APP=app
export FLASK_ENV=development

# Run the development server
flask run
# API available at http://127.0.0.1:5000/api/v1
```

Folder Structure

```
taskflow-api/
├── app/
│   ├── __init__.py      # application factory
│   ├── models/
│   ├── routes/
│   ├── services/
│   └── schemas/
├── migrations/          # Alembic migration scripts
├── tests/
├── config.py
├── requirements.txt
├── Dockerfile
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Swagger / API Docs | Sample Request | Sample Response |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Add WebSocket support for real-time task updates
- Introduce Celery for background email notifications
- Publish OpenAPI/Swagger documentation via Flask-Smorest

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Web Development Project README

For full-stack MERN / Next.js style application repositories.

Sample Repository: "ShopLite - Full-Stack E-Commerce Platform"

Project Overview

A full-stack e-commerce application supporting product browsing, cart management, secure checkout, and an admin dashboard for inventory control, built on the MERN stack with a responsive, accessible UI.

Features

- Responsive storefront with product search, filters, and infinite scroll
- Secure authentication (JWT + bcrypt) and persistent cart via local state sync
- Stripe-integrated checkout flow with order confirmation emails
- Admin dashboard for product, order, and inventory management
- Lighthouse performance score of 95+ on mobile and desktop

Tech Stack

React • Redux Toolkit • Node.js • Express • MongoDB • Tailwind CSS • Stripe API • Vercel / Render

Architecture

- Client - React SPA with Redux Toolkit for state, React Router for navigation (client/)
- Server - Express REST API with layered controllers/services/models (server/)
- Database - MongoDB collections for users, products, orders (server/models/)
- Payments - Stripe webhook handler for order confirmation (server/webhooks/)

Installation

```
git clone https://github.com/YOUR_USERNAME/shoplite.git
cd shoplite
cd server && npm install && cd ../client && npm install
```

Usage

```
# Run backend
cd server && npm run dev

# Run frontend (separate terminal)
cd client && npm run dev
# App available at http://localhost:5173
```

Folder Structure

```
shoplite/
├── client/             # React front-end
```

```
│   ├── src/
│   │   ├── components/
│   │   ├── pages/
│   │   ├── redux/
│   │   └── App.jsx
│   └── package.json
├── server/             # Express back-end
│   ├── controllers/
│   ├── models/
│   ├── routes/
│   ├── webhooks/
│   └── server.js
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Storefront | Product Page | Checkout Flow |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Add product recommendation engine
- Migrate client build to Next.js for SSR and SEO gains
- Add multi-vendor marketplace support

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Python Project README

For CLI tools, automation scripts, and general-purpose Python libraries.

Sample Repository: "PyOrganize - Smart File Organizer CLI"

Project Overview

A cross-platform command-line tool that automatically sorts, renames, and archives files by type, date, or custom rules, designed to be installed as a pip package and scheduled as a background job.

Features

- Rule-based sorting engine configurable via a single YAML file
- Dry-run mode that previews changes before touching the filesystem
- Duplicate detection using file hashing (SHA-256)
- Cross-platform support (Windows, macOS, Linux) with no external dependencies
- Published to PyPI with semantic-versioned releases

Tech Stack

Python 3.11 • argparse / Click • PyYAML • Pytest • GitHub Actions • PyPI (twine)

Architecture

- CLI Layer - argument parsing and command dispatch (pyorganize/cli.py)
- Core Logic - sorting rules engine and hashing utilities (pyorganize/core/)
- Config - YAML rule loader with schema validation (pyorganize/config.py)
- Packaging - setup.cfg / pyproject.toml for PyPI distribution

Installation

```
pip install pyorganize

# or, for local development:
git clone https://github.com/YOUR_USERNAME/pyorganize.git
cd pyorganize && pip install -e .
```

Usage

```
# Preview changes without moving files
pyorganize sort ~/Downloads --dry-run

# Apply sorting using a custom rule file
pyorganize sort ~/Downloads --config rules.yaml
```

Folder Structure

```
pyorganize/
```

```
├── pyorganize/
│   ├── cli.py
│   ├── config.py
│   └── core/
│       ├── sorter.py
│       └── hasher.py
├── tests/
├── pyproject.toml
├── rules.example.yaml
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| CLI Help Output | Dry-Run Preview | Applied Result |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Add a watch-mode daemon for real-time organizing
- Add cloud storage support (Google Drive, Dropbox)
- Ship a Textual-based terminal UI

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Android Project README

For native Kotlin / Jetpack Compose mobile application repositories.

Sample Repository: "StudyBuddy - Offline-First Study Planner"

Project Overview

A native Android application that helps students plan revision schedules, track study streaks, and receive smart reminders, built with Jetpack Compose and an offline-first Room database architecture.

Features

- Offline-first architecture using Room with background sync placeholder
- Jetpack Compose UI following Material 3 design guidelines
- Smart notification scheduling via WorkManager
- Study streak tracking with animated progress indicators
- Dark mode and dynamic theming (Material You) support

Tech Stack

Kotlin • Jetpack Compose • Room • WorkManager • Hilt (DI) • MVVM • Material 3

Architecture

- UI Layer - Composable screens observing ViewModel state (app/ui/)
- Domain Layer - use-cases encapsulating business rules (app/domain/)
- Data Layer - Room DAOs and repository implementations (app/data/)
- DI - Hilt modules wiring dependencies across layers (app/di/)

Installation

```
git clone https://github.com/YOUR_USERNAME/studybuddy-android.git
cd studybuddy-android
# Open in Android Studio (Hedgehog or newer)
# Let Gradle sync, then run on an emulator or device
```

Usage

```
# Build a debug APK from the command line
./gradlew assembleDebug

# Run unit tests
./gradlew test
```

Folder Structure

```
studybuddy-android/
├── app/
```

```
│   ├── src/main/java/com/studybuddy/
│   │   ├── ui/
│   │   ├── domain/
│   │   ├── data/
│   │   └── di/
│   └── src/main/res/
├── build.gradle.kts
├── settings.gradle.kts
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Home Screen | Study Streak Tracker | Settings / Theme |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Future Improvements

- Add cloud backup via Firebase Firestore
- Introduce collaborative study groups feature
- Add widget support for home-screen streak tracking

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

College Major Project README

For final-year / capstone submissions requiring an academic tone and thorough documentation.

Sample Repository: "Smart Attendance System using Facial Recognition"

Project Overview

A final-year capstone project that automates classroom attendance using real-time facial recognition, reducing manual roll-call time and producing auditable digital attendance records for institutional use.

Features

- Real-time face detection and recognition using OpenCV and a FaceNet embedding model
- Automated attendance logging to a relational database with timestamped entries
- Admin panel for student enrolment, class scheduling, and report export (CSV/PDF)
- Liveness check to mitigate photo-spoofing attempts
- Achieved 94.7% recognition accuracy across a 120-student test dataset

Tech Stack

Python • OpenCV • FaceNet / dlib • Flask • SQLite • Bootstrap

Architecture

- Capture Module - webcam frame capture and face-detection pipeline (src/capture.py)
- Recognition Module - embedding generation and identity matching (src/recognize.py)
- Persistence - SQLite schema for students, sessions, and attendance logs (src/db.py)
- Admin Interface - Flask + Bootstrap dashboard for staff (app.py, templates/)

Installation

```
git clone https://github.com/YOUR_USERNAME/smart-attendance-system.git
cd smart-attendance-system
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
```

Usage

```
# Enrol new students (captures reference face embeddings)
python src/enrol.py --name "Student Name" --id 2026CS101

# Start attendance session
python app.py
```

Folder Structure

```
smart-attendance-system/
├── src/
```

```
│   ├── capture.py
│   ├── recognize.py
│   ├── enrol.py
│   └── db.py
├── templates/            # admin dashboard views
├── static/
├── docs/                 # project report, diagrams, viva PPT
├── requirements.txt
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Student Enrolment | Live Recognition | Attendance Report |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Academic Context

Submitted in partial fulfilment of the requirements for the degree of Bachelor of Technology, under the guidance of a designated faculty supervisor. Include the department, university name, guide's name, and academic year directly beneath this heading in your final report and repository README.

Future Improvements

- Migrate storage from SQLite to PostgreSQL for multi-department scale
- Add mobile companion app for staff to mark manual overrides
- Integrate with existing college ERP via REST API

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

Hackathon Project README

For time-boxed builds - optimized for judges skimming quickly under a ticking clock.

Sample Repository: "ReliefLink - Disaster Resource Coordination App"

Project Overview

Built in 30 hours at HackNation 2026, ReliefLink connects disaster-affected individuals with nearby verified relief camps, live supply availability, and volunteer coordination through a real-time map interface.

Features

- Live relief-camp map with real-time supply-level indicators (built in the 30-hour window)
- SOS request form routing to the nearest camp with available capacity
- Volunteer sign-up and task-assignment board
- Fully functional MVP demoed live to judges with seeded demo data

Tech Stack

React • Firebase (Firestore + Auth) • Google Maps API • Tailwind CSS

Architecture

- Client - React app with Firebase SDK for real-time data sync (src/)
- Backend-as-a-Service - Firestore collections for camps, requests, volunteers
- Maps - Google Maps JavaScript API for live camp visualization

Installation

```
git clone https://github.com/YOUR_USERNAME/relieflink.git
cd relieflink
npm install
# add your Firebase config to src/firebase.js
```

Usage

```
npm run dev
# App available at http://localhost:5173
```

Folder Structure

```
relieflink/
├── src/
│   ├── components/
│   ├── pages/
│   ├── firebase.js
│   └── App.jsx
├── public/
├── package.json
```

```
└── README.md
```

Screenshots

```
## 🖼️ Screenshots

| Live Camp Map | SOS Request Form | Volunteer Board |
|:---:|:---:|:---:|
| ![Screenshot 1](docs/screenshots/screen-1.png) | ![Screenshot
2](docs/screenshots/screen-2.png) | ![Screenshot 3](docs/screenshots/screen-3.png) |
```

Hackathon Context

Team: [Team Name] · Event: [Hackathon Name, Year] · Track: [Track Name] · Result: [e.g. "Top 10 Finalist"]. State the build window explicitly (e.g. "built in 30 hours") - judges and recruiters both weight execution speed heavily, and honesty about scope builds credibility rather than costing it.

Future Improvements

- Add SMS-based SOS submission for low-connectivity areas
- Integrate government disaster-alert data feeds
- Harden data model and add authentication before any production use

License

This project is licensed under the MIT License - see the LICENSE file for details.

Author

Created by Ansh Banga - LinkedIn: linkedin.com/in/ansh-banga-775275330

PART THREE

Portfolio Repository Structure

A dedicated repository that indexes and narrates your best work - the bridge between your profile and your projects.

Beyond the profile README, a dedicated portfolio repository (commonly named portfolio, or the source for a personal site deployed via GitHub Pages / Vercel) gives you a controlled, narrative space to present 4-6 flagship projects in depth.

Think of it as the difference between a resume bullet and a case study.

3.1 Recommended Structure

```
portfolio/
├── public/                 # static assets (favicon, resume.pdf, og-image.png)
├── src/
│   ├── components/         # Navbar, ProjectCard, ContactForm, etc.
│   ├── data/
│   │   └── projects.json   # single source of truth for project metadata
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Projects.jsx
│   │   ├── About.jsx
│   │   └── Contact.jsx
│   ├── assets/              # images, icons
│   └── App.jsx
├── .env.example
├── package.json
└── README.md
```

3.2 What Belongs on Each Page

Page	Must Include	Avoid
Home	Name, role, one-line value proposition, primary CTA	Long paragraphs, auto-playing audio
Projects	3-6 projects, outcome-first descriptions, live + code links	Every project you've ever pushed
About	Career narrative, skills, education, timeline	Unverifiable claims, filler adjectives
Contact	Email, LinkedIn, resume download, response-time expectation	Contact forms with no confirmation state

RECRUITER INSIGHT

List projects by relevance to the role you're targeting, not strictly by date. A recruiter hiring for backend roles should see your API project first, even if your newest project is a front-end experiment.

Page	Must Include	Avoid

PRO TIP

Keep project descriptions outcome-first: “Reduced API response time by 40% by introducing Redis caching” beats “Built a caching layer.”

PART FOUR

Professional Repository Structure

The conventions that separate a repository that looks maintained from one that looks abandoned.

Before a recruiter or reviewer reads a single line of code, the shape of your repository has already told them how you work.

Consistent naming, sensible folder layout, a deliberate branching strategy, and disciplined versioning are the four pillars covered in this chapter.

4.1 Naming Conventions

Element	Convention	Example
Repository name	kebab-case, descriptive, no personal shorthand	expense-tracker-api
Branch name	type/short-description	feature/user-authentication
File name (Python)	snake_case.py	data_loader.py
File name (JS/React)	PascalCase.jsx for components, camelCase.js for utils	ProjectCard.jsx, formatDate.js
Environment file	.env.example committed, .env gitignored	.env.example
Commit tag / release	Semantic Versioning	v1.4.0

4.2 Standard Folder Organization

```
project-root/
├── .github/
│   ├── workflows/          # CI/CD pipelines
│   └── ISSUE_TEMPLATE/
├── docs/                   # architecture notes, diagrams, ADRs
├── src/                    # application source code
├── tests/                  # unit, integration, e2e tests
├── scripts/                # one-off automation scripts
├── .env.example
├── .gitignore
├── CONTRIBUTING.md
├── CODE_OF_CONDUCT.md
├── LICENSE
└── README.md
```

4.3 Branching Strategy

For most student and small-team projects, a simplified GitHub Flow outperforms heavier strategies like GitFlow - fewer long-lived branches means fewer merge conflicts and a lower cognitive load.

Branch	Purpose	Lifetime
main	Always deployable / demo-ready	Permanent
develop (optional, larger teams)	Integration branch before release	Permanent
feature/*	One feature or task per branch	Deleted after merge
fix/*	Bug fixes	Deleted after merge
release/*	Release stabilization / final QA	Deleted after tag
hotfix/*	Urgent production fix branched from main	Deleted after merge

BEST PRACTICE

Protect your main branch: require pull requests, at least one review, and passing CI checks before merge. Even solo projects benefit - it forces you to review your own diffs before they land.

4.4 Versioning: Semantic Versioning (SemVer)

Tag releases as MAJOR.MINOR.PATCH - for example v2.1.0. Increment MAJOR for breaking changes, MINOR for backward-compatible features, and PATCH for backward-compatible fixes.

Segment	Increment When	Example
MAJOR	Breaking API/behavior change	v1.x.x → v2.0.0
MINOR	New backward-compatible feature	v2.0.0 → v2.1.0
PATCH	Backward-compatible bug fix	v2.1.0 → v2.1.1

4.5 Best Practices Checklist

- One repository, one clear purpose - avoid monorepos for student projects unless explicitly required
- A README that renders correctly before the first commit is even pushed
- Secrets and credentials never committed - always via .env and a documented .env.example
- CI configured early, not retrofitted after the project is “done”
- Issues used for planning, even solo - it builds a visible, traceable history of decisions

⚠ COMMON MISTAKE

Committing node_modules/, venv/, .env, or build artifacts is the single most common repository hygiene failure reviewers flag. Fix your .gitignore before your first commit, not after your tenth.

PART FIVE

The Complete Git Guide

Every command you need for daily development work, explained with the practical use case behind it - not just the syntax.

Git is a distributed version control system: every clone contains the full history of the project, which is what makes branching, merging, and offline work possible in the first place. The fourteen commands below cover well over 95% of day-to-day Git usage.

git init

Purpose: Initializes a new, empty Git repository in the current directory by creating a hidden .git subfolder that stores all version history.

When to Use: Use this exactly once, at the very start of a new local project - before your first commit.

Example:

```
git init
git init my-project        # creates a new folder and initializes it
```

git clone

Purpose: Downloads a complete copy of a remote repository, including its full commit history and all branches, into a new local folder.

When to Use: Use this whenever you're starting work on an existing project - your own, a teammate's, or an open-source project you intend to contribute to.

Example:

```
git clone https://github.com/USERNAME/REPO.git
git clone https://github.com/USERNAME/REPO.git custom-folder-name
```

git add

Purpose: Stages changes - moving them from your working directory into the staging area, in preparation for a commit.

When to Use: Use this after editing files, to select precisely what should be included in your next commit (a good habit for atomic, reviewable commits).

Example:

```
git add file.py
git add .                  # stage everything in the current directory
git add -p                 # stage changes interactively, hunk by hunk
```

git commit

Purpose: Saves a permanent snapshot of everything currently staged, along with a message describing the change.

When to Use: Use this after staging a logically complete, working change - not after every single line edit, and not as one giant dump at the end of the day.

Example:

```
git commit -m "feat: add JWT refresh token rotation"
git commit -am "fix: correct off-by-one in pagination"   # stage tracked files +
commit
```

git push

Purpose: Uploads your local commits to a remote repository (typically GitHub), making them visible to collaborators.

When to Use: Use this after committing locally, whenever you're ready to share progress, open a pull request, or trigger CI.

Example:

```
git push origin main
git push -u origin feature/login-page   # sets upstream so future 'git push' needs no
args
```

git pull

Purpose: Fetches new commits from the remote and immediately merges them into your current local branch (equivalent to git fetch followed by git merge).

When to Use: Use this at the start of every work session and before pushing, to make sure you're building on the latest shared history.

Example:

```
git pull origin main
git pull --rebase origin main   # replay your local commits on top instead of merging
```

git fetch

Purpose: Downloads new commits and branches from the remote without merging them into your current branch - it only updates your local view of the remote.

When to Use: Use this when you want to inspect what changed upstream before deciding how (or whether) to integrate it.

Example:

```
git fetch origin
git log origin/main --oneline -5   # inspect remote history before merging
```

git checkout

Purpose: Switches between branches or restores files to a previous state. (Modern Git splits this into git switch for branches and git restore for files, but checkout remains near-universal.)

When to Use: Use this to move between feature branches, or to create a new branch and switch to it in one step.

Example:

```
git checkout main
git checkout -b feature/dark-mode   # create and switch to a new branch
git checkout -- file.py             # discard local changes to a file
```

git merge

Purpose: Integrates the changes from one branch into another, creating a merge commit when the histories have diverged.

When to Use: Use this to bring a completed feature branch back into main, or to update a feature branch with the latest main.

Example:

```
git checkout main
git merge feature/dark-mode
```

git rebase

Purpose: Replays your branch's commits on top of another branch's latest history, producing a linear, easier-to-read commit log instead of a merge commit.

When to Use: Use this to keep a feature branch up to date with main before opening a pull request - but avoid rebasing commits that have already been pushed and shared.

Example:

```
git checkout feature/dark-mode
git rebase main
git rebase -i HEAD~3    # interactive rebase to squash/reorder the last 3 commits
```

git stash

Purpose: Temporarily shelves uncommitted changes so you can switch context (e.g. to fix an urgent bug) without committing half-finished work.

When to Use: Use this when you need to switch branches immediately but aren't ready to commit what you're working on.

Example:

```
git stash
git stash save "WIP: cart total calculation"
git stash list
git stash pop        # reapply and remove the most recent stash
```

git reset

Purpose: Moves the current branch pointer to a different commit, optionally modifying the staging area and working directory (--soft, --mixed, --hard).

When to Use: Use --soft to undo a commit but keep changes staged; use --hard only when you are certain you want to discard changes entirely.

Example:

```
git reset --soft HEAD~1    # undo last commit, keep changes staged
git reset --hard HEAD~1    # discard last commit and its changes completely
```

git revert

Purpose: Creates a new commit that undoes the changes introduced by a previous commit, without rewriting history.

When to Use: Use this on shared/public branches to safely undo a bad commit - unlike reset, it preserves history, which is essential once a commit has been pushed.

Example:

```
git revert a1b2c3d
git revert --no-commit a1b2c3d   # stage the revert without committing immediately
```

git tag

Purpose: Marks a specific commit as a named reference point, most commonly used to mark release versions.

When to Use: Use this immediately after merging a release branch, to create a permanent, citable marker for that version.

Example:

```
git tag v1.0.0
git tag -a v1.0.0 -m "First stable release"
git push origin v1.0.0
```

BEST PRACTICE

Write commit messages before you write the code, not after. Framing the intent first (“fix: correct pagination off-by-one”) keeps the diff focused on exactly one problem.

⚠ COMMON MISTAKE

Never rebase or force-push a branch that others have already pulled from. It rewrites shared history and breaks everyone else's local copy.

REMEMBER

git reset rewrites history; git revert adds to it. On any branch other than your own private feature branch, prefer revert.

RECRUITER INSIGHT

> Interviewers frequently ask candidates to walk through their own Git history. Being able to explain why you chose rebase over merge on a given branch signals real fluency, not memorized commands.

PART SIX

Conventional Commit Guide

A commit history that reads like a changelog is a competitive advantage - this is the specification recruiters and CI tools both recognize.

The Conventional Commits specification structures each commit message as type(scope): description. Beyond readability, this format enables automated changelog generation and semantic-version bumping - many CI pipelines parse it directly.

```
<type>(<optional scope>): <short description>

<optional longer body>

<optional footer - e.g. BREAKING CHANGE:, Closes #42>
```

6.1 Commit Types Reference

Type	When to Use	Example
feat	A new feature for the user	feat(auth): add Google OAuth login option
fix	A bug fix	fix(cart): correct total price rounding error
docs	Documentation-only changes	docs(readme): add installation steps for Windows
style	Formatting only - no logic change (whitespace, semicolons)	style(header): apply Prettier formatting to Nav.jsx
refactor	Code change that neither fixes a bug nor adds a feature	refactor(api): extract validation logic into middleware
perf	A code change that improves performance	perf(query): add index to reduce lookup time by 60%
test	Adding or correcting tests	test(auth): add unit tests for token refresh flow
build	Changes to build system or external dependencies	build(deps): upgrade react from 18.2.0 to 18.3.1
chore	Routine tasks that don't modify src or test files	chore(release): bump version to v1.4.0

6.2 Writing the Description

- Use the imperative mood: “add” not “added” or “adds” - read it as completing the sentence “This commit will...”
- Keep the first line under 72 characters so it doesn't wrap in GitHub's UI or terminal log views
- Don't end the description with a period
- Reference the related issue in the footer: Closes #23

PRO TIP

A BREAKING CHANGE: footer (or a ! after the type, e.g. feat!:) signals a major version bump to any tooling that parses your commit history automatically.

⚠ COMMON MISTAKE

Avoid vague commits like “fix stuff” or “update”. A commit message should let a reviewer understand the change without opening the diff.

RECRUITER INSIGHT

A technical interviewer who opens your `git log` before your code is not unusual. A clean, Conventional-Commit-formatted history signals process discipline in the first ten seconds of a repo review.

PART SEVEN

The Complete GitHub Workflow

From a raw idea to a tagged, released version - the end-to-end path every professional repository follows.

This is the workflow to internalize until it's automatic. Each stage below builds on the previous one; skipping a stage is almost always where avoidable bugs and messy histories come from.

7.1 The Eight-Stage Workflow

1. Idea

Define the problem in one sentence and identify who it's for. Write it down - a one-paragraph spec - before opening an editor.

2. Repository

Create the GitHub repository first, with a README, .gitignore, and LICENSE selected at creation time. Clone it locally rather than initializing locally and connecting later.

3. Development

Work in short-lived feature branches off main. Commit early and often, in small, logically complete units.

4. Commit

Stage deliberately (git add -p when needed) and write a Conventional Commit message for every change.

5. Push

Push your feature branch regularly - daily at minimum - so work is backed up and visible, and CI can run.

6. Pull Request

Open a PR against main with a clear description, linked issues, and screenshots/GIFs for any UI change. Request review even if working solo, by self-reviewing the diff.

7. Merge

Once checks pass and review is approved, merge using “Squash and merge” for feature branches to keep main's history clean, then delete the branch.

8. Release

Tag a semantic version (e.g. v1.2.0), write release notes summarizing user-facing changes, and publish a GitHub Release.

> 🚀 **BEST PRACTICE**
>
> Treat main as production. Nothing lands there without passing through a pull request, even on a one-person project - the discipline transfers directly to team environments.

RECRUITER INSIGHT

A repository with a visible history of small commits, linked issues, and merged pull requests demonstrates process maturity that a single “initial commit” + “final version” repo cannot.

PART EIGHT

Open Source Contribution Guide

One merged pull request to a real project outweighs ten toy repositories on a resume.

Open-source contribution is one of the highest-signal activities a student can put on a resume - it proves you can read unfamiliar code, follow project conventions, and collaborate asynchronously with maintainers you've never met.

The seven-step flow below is the same regardless of project size.

1. Fork

Create your own copy of the repository under your GitHub account. This gives you a sandbox to work in without requiring write access to the original project.

```
# On GitHub: click "Fork" on the project page
# This creates https://github.com/YOUR_USERNAME/project-name
```

2. Clone

Download your fork locally, and add the original repository as a second remote (conventionally named upstream) so you can pull in future updates.

```
git clone https://github.com/YOUR_USERNAME/project-name.git
cd project-name
git remote add upstream https://github.com/ORIGINAL_OWNER/project-name.git
```

3. Branch

Never work directly on main. Create a descriptively named branch for the specific issue or feature you're addressing.

```
git checkout -b fix/issue-142-null-pointer
```

4. Commit

Make focused commits using Conventional Commit messages, and keep your branch synced with upstream/main as you work to minimize merge conflicts later.

```
git fetch upstream
git rebase upstream/main
git add .
git commit -m "fix(parser): handle null input in tokenizer"
```

5. Pull Request

Push your branch to your fork and open a pull request against the original repository. Reference the issue number, describe what changed and why, and include before/after evidence for visual changes.

```
git push origin fix/issue-142-null-pointer
# On GitHub: click "Compare & pull request"
```

6. Review

Respond to maintainer feedback promptly and without defensiveness - treat requested changes as the normal, expected part of the process, not as criticism of your ability.

7. Merge

Once approved, a maintainer merges your PR. Your commit becomes part of permanent project history under your GitHub identity - a durable, verifiable credential.

RECRUITER INSIGHT

Recruiters and hiring managers can click through to your actual merged PRs on real projects. This is verifiable proof of skill in a way that a resume bullet never can be.

PRO TIP

Start with issues labeled “good first issue” or “help wanted.” They're explicitly scoped by maintainers to be approachable for new contributors.

⚠ COMMON MISTAKE

Opening a pull request with an unrelated, project-wide reformat mixed into a small fix is one of the fastest ways to frustrate a maintainer. Keep PRs scoped to one concern.

PART NINE

Ready-to-Use Templates

Copy-paste-ready files that instantly make any repository look actively maintained.

Add these directly to your repository - issue templates go in .github/ISSUE_TEMPLATE/, the pull request template goes in .github/PULL_REQUEST_TEMPLATE.md, and CONTRIBUTING.md / CODE_OF_CONDUCT.md sit at the repository root.

Bug Report Template

```
---
name: Bug Report
about: Report a reproducible problem
title: "[BUG] "
labels: bug
---

**Describe the bug**
A clear, concise description of what the bug is.

**To Reproduce**
1. Go to '...'
2. Click on '...'
3. See error

**Expected behavior**
What you expected to happen instead.

**Screenshots**
If applicable, add screenshots to help explain the problem.

**Environment:**
- OS: [e.g. Windows 11]
- Browser/Version: [e.g. Chrome 126]
- App Version: [e.g. v1.3.0]

**Additional context**
Any other context about the problem.
```

Feature Request Template

```
---
name: Feature Request
about: Suggest a new feature or enhancement
title: "[FEATURE] "
labels: enhancement
---
```

```
**Is your feature request related to a problem?**
A clear description of the problem, e.g. "I'm frustrated when..."

**Describe the solution you'd like**
A clear description of what you want to happen.

**Describe alternatives you've considered**
Any alternative solutions or features you've considered.

**Additional context**
Mockups, references, or related issues.
```

Question Template

```
---
name: Question
about: Ask a question about usage or behavior
title: "[QUESTION] "
labels: question
---

**What are you trying to do?**

**What have you already tried?**

**Relevant code / configuration (if any)**
```

Documentation Issue Template

```
---
name: Documentation Issue
about: Report unclear, missing, or incorrect documentation
title: "[DOCS] "
labels: documentation
---

**Which page/section is affected?**
Link or file path.

**What is unclear, missing, or incorrect?**

**Suggested improvement**
Proposed wording or structure, if you have one.
```

Pull Request Template

```
## Description
Briefly describe what this PR changes and why.

## Related Issue
Closes #

## Type of Change
- [ ] Bug fix
- [ ] New feature
- [ ] Breaking change
- [ ] Documentation update

## How Has This Been Tested?
Describe the tests you ran and how to reproduce them.

## Checklist
- [ ] My code follows this project's style guidelines
- [ ] I have performed a self-review of my own code
- [ ] I have added tests that prove my fix/feature works
- [ ] I have updated relevant documentation
```

CONTRIBUTING.md Template

```
# Contributing to [Project Name]

Thank you for considering a contribution! Please follow the steps below.

## Getting Started
1. Fork the repository and clone your fork locally.
2. Install dependencies: `npm install` (or `pip install -r requirements.txt`).
3. Create a branch: `git checkout -b feature/your-feature-name`.

## Development Guidelines
- Follow the existing code style (run the linter before committing).
- Write tests for any new functionality.
- Keep pull requests focused on a single concern.

## Commit Messages
This project follows the Conventional Commits specification
(e.g. `feat: add dark mode toggle`).

## Submitting a Pull Request
1. Ensure all tests pass locally.
2. Push your branch and open a PR against `main`.
3. Fill out the PR template completely and link any related issues.

## Code of Conduct
This project adheres to a Code of Conduct. By participating, you are
```

```
expected to uphold it - see CODE_OF_CONDUCT.md.
```

CODE_OF_CONDUCT.md Template

```
# Contributor Code of Conduct

## Our Pledge
We as members, contributors, and maintainers pledge to make participation
in our project a harassment-free experience for everyone, regardless of
age, body size, disability, ethnicity, gender identity and expression,
experience level, nationality, personal appearance, race, religion, or
sexual identity and orientation.

## Our Standards
Examples of behavior that contributes to a positive environment:
- Using welcoming and inclusive language
- Being respectful of differing viewpoints and experiences
- Gracefully accepting constructive criticism

Examples of unacceptable behavior:
- Trolling, insulting/derogatory comments, and personal or political attacks
- Public or private harassment
- Publishing others' private information without explicit permission

## Enforcement
Instances of abusive, harassing, or otherwise unacceptable behavior may
be reported to the project maintainers at [contact email]. All complaints
will be reviewed and investigated promptly and fairly.

## Attribution
Adapted from the Contributor Covenant, version 2.1.
```

BEST PRACTICE

Repositories with all seven templates in place signal a maintenance-ready project even on day one - this alone differentiates a student repository from 90% of others.

PART TEN

LICENSE Guide

An unlicensed repository is, by default, all rights reserved - nobody may legally use, copy, or modify it.

Choosing a license is not a formality; without one, default copyright law means others technically cannot use your code even if it's public on GitHub. This chapter covers the four licenses students and early-career engineers encounter most often.

License	Permissions	Conditions	Best For
MIT	Commercial use, modification, distribution, private use	Include original license & copyright notice	Personal projects, libraries, student portfolios wanting maximum adoption
Apache 2.0	Same as MIT, plus explicit patent grant	Include license, state changes made, retain notices	Projects concerned about patent protection, larger corporate-adjacent contributions
GPL v3	Commercial use, modification, distribution	Derivative works must also be open-sourced under GPL ("copyleft")	Projects that want all future forks to remain open source
BSD 3-Clause	Commercial use, modification, distribution, private use	Include license & copyright; cannot use contributors' names to promote derivatives	Academic and research-oriented projects

10.1 Choosing the Right One

- Want the widest possible adoption with the fewest restrictions? Choose MIT.
- Worried about patent disputes from contributors or users? Choose Apache 2.0.
- Want to guarantee that anyone who builds on your work also open-sources their version? Choose GPL v3.
- Publishing academic or research code where attribution matters more than restriction? Choose BSD 3-Clause.

PRO TIP

For 90% of student portfolio projects, MIT is the right default - it maximizes the chance a recruiter or future employer can freely reference or reuse your code as a work sample.

REMEMBER

Always add a LICENSE file at the repository root, not just a mention in the README - GitHub only displays the license badge automatically when it detects a proper LICENSE file.

PART ELEVEN

.gitignore Examples

Every stack has its own set of files that should never reach version control.

A missing or incomplete .gitignore is one of the fastest ways to bloat a repository with build artifacts, dependencies, and secrets. Use the stack-specific examples below as a starting point - add project-specific entries as needed.

Python

```
__pycache__/
*.py[cod]
*.egg-info/
venv/
.venv/
env/
.env
.pytest_cache/
.mypy_cache/
*.log
dist/
build/
```

Node.js

```
node_modules/
npm-debug.log*
yarn-debug.log*
yarn-error.log*
dist/
build/
.env
.env.local
coverage/
.DS_Store
```

Flask

```
__pycache__/
*.pyc
venv/
.env
instance/
*.sqlite3
.flaskenv
.pytest_cache/
```

Java

```
*.class
*.jar
*.war
target/
build/
.gradle/
.idea/
*.iml
out/
```

React

```
node_modules/
build/
dist/
.env
.env.local
*.log
coverage/
.DS_Store
.vscode/
```

Android

```
*.iml
.gradle/
/local.properties
/.idea/
/build/
app/build/
*.apk
*.ap_
*.dex
.DS_Store
captures/
```

⚠ COMMON MISTAKE

If secrets were already committed before you added them to .gitignore, adding the rule alone does not remove them from history - you must also purge them (e.g. with git filter-repo) and rotate the exposed credentials.

PART TWELVE

GitHub Best Practices

The habits that separate a placement-ready profile from a repository that merely exists.

🎯 Recruiter Tips

- Pin 4-6 repositories that best represent the roles you're targeting - not simply your most recent work
- Make sure every pinned repository's README renders cleanly with no broken image links
- Use your real name and a professional profile photo - recruiters cross-reference GitHub with LinkedIn
- Link your resume and LinkedIn directly from your profile README, not buried three clicks deep

📁 Portfolio Tips

- Lead every project description with the outcome, not the implementation
- Include a live demo link wherever technically possible - deployed beats “clone to run locally”
- Keep 3-5 flagship projects, each with real depth, rather than 20 shallow ones

📦 Repository Tips

- Every repository needs: README, LICENSE, .gitignore, and a clear folder structure, at minimum
- Squash noisy work-in-progress commits before merging to keep main's history readable
- Add topics/tags to your repository (e.g. python, flask, machine-learning) - GitHub's search and recruiters both use them

👤 Profile Tips

- Keep your contribution graph consistent rather than bursty - little and often reads better than one binge week per semester
- Write a bio that states your specialization, not just “student” or “coder”
- Pin your profile README repository update date mentally - revisit and refresh it every semester

🎓 Placement Tips

- Tailor which projects are pinned to the specific companies/roles you're applying to that cycle
- Be ready to explain any architectural decision in your pinned projects in an interview - depth beats breadth
- Include your GitHub link directly on your resume header, not just LinkedIn

⚠ Common Mistakes

- Uploading assignment solutions or tutorial-clone code without attribution or added original value
- Leaving default README.md (“# project-name”) on public repositories
- Committing .env files, API keys, or database credentials

- Force-pushing over shared branches without coordinating with collaborators
- Having 100+ repositories with no pinned projects and no profile README - signal is lost in noise

> **BEST PRACTICE**
>
> Audit your entire GitHub profile once per semester using this checklist. A five-minute audit before applications open can be the difference between a recruiter clicking through or bouncing.

BONUS CHAPTER

Top 20 GitHub Projects Every Engineering Student Should Build

A curated ladder of projects, ordered by difficulty, designed to build a portfolio that tells a coherent growth story.

The strongest student GitHub profiles don't show twenty disconnected experiments - they show a visible progression from fundamentals to systems thinking.

Use this list as a roadmap, not a checklist to rush through. Build fewer projects, more deeply, and document each one using the README templates from Part Two.

Beginner Projects (Build Confidence & Fundamentals)

#	Project	Description	Recommended Stack
1	Personal Portfolio Website	A responsive single-page site presenting your projects, skills, and contact info.	HTML, CSS, JavaScript / React, Tailwind CSS
2	To-Do List with Local Persistence	A CRUD task manager that persists tasks between sessions.	React, Context API / Redux, LocalStorage
3	Weather App	Fetches and displays live weather using a public API with search by city.	JavaScript, OpenWeather API, Fetch/Axios
4	Expense Tracker	Logs income and expenses with category breakdowns and simple charts.	Python/Flask or React, SQLite, Chart.js
5	Markdown Notes App	A note-taking app that renders Markdown live as the user types.	React, Marked.js, LocalStorage
6	Quiz / Trivia App	A timed multiple-choice quiz app with score tracking.	JavaScript, Trivia API, CSS animations

Intermediate Projects (Full-Stack & Systems Thinking)

#	Project	Description	Recommended Stack
7	Blog Platform with Auth	A multi-user blogging platform with authentication, rich-text posts, and comments.	MERN Stack, JWT, Cloudinary
8	E-Commerce Storefront	Product catalog, cart, and checkout flow with a payment gateway integration.	React, Node.js, MongoDB, Stripe API
9	Real-Time Chat Application	A multi-room chat app with typing indicators and message persistence.	Socket.IO, Node.js, Express, MongoDB
10	REST API with Authentication	A well-documented, role-based API with rate limiting and automated tests.	Flask/Django REST Framework, PostgreSQL, Pytest
11	Movie Recommendation System	Content-based recommender using cosine similarity on movie metadata.	Python, Pandas, Scikit-learn, Streamlit

#	Project	Description	Recommended Stack
12	Android Expense Manager	A native offline-first personal finance tracker with charts.	Kotlin, Jetpack Compose, Room
13	Job Board Aggregator	Scrapes and normalizes job listings from multiple sources into a searchable UI.	Python, BeautifulSoup/Scrapy, Flask, PostgreSQL

Advanced Projects (Placement & Research-Grade)

#	Project	Description	Recommended Stack
14	Distributed URL Shortener	A horizontally scalable URL shortener with analytics and caching layer.	Node.js/Go, Redis, PostgreSQL, Docker, Nginx
15	Real-Time Collaborative Code Editor	Multiple users editing the same document simultaneously with conflict resolution.	React, Node.js, WebSockets, Operational Transform / CRDT
16	End-to-End MLOps Pipeline	Automated training, versioning, and deployment pipeline for an ML model.	Python, MLflow, Docker, GitHub Actions, AWS/GCP
17	Microservices-Based Food Delivery Backend	Independently deployable services for orders, payments, and delivery tracking.	Node.js/Java (Spring Boot), Docker, Kubernetes, RabbitMQ
18	Custom Object Detection System	Trains and serves a custom YOLO/SSD model for a niche detection task.	Python, PyTorch, OpenCV, FastAPI
19	Blockchain-Based Voting System	A permissioned smart-contract application simulating tamper-proof voting.	Solidity, Hardhat, Ethers.js, React
20	Full-Stack SaaS Analytics Dashboard	Multi-tenant analytics platform with subscription billing.	Next.js, PostgreSQL, Prisma, Stripe Billing, Vercel

RECRUITER INSIGHT

In interviews, one advanced project you can explain end-to-end - including the trade-offs you rejected - outweighs seven beginner projects listed without context.

BEST PRACTICE

Pick one project per tier that intersects with your target job role (e.g. an ML student should prioritize the recommendation system and the MLOps pipeline) and polish those to publication quality using Part Two's templates.

REMEMBER

A project isn't finished when the code runs - it's finished when the README, tests, and license are in place and a stranger could clone it and understand it in five minutes.

END OF HANDBOOK

The Complete GitHub Starter Pack - 2026 Edition

Build in public. Document with intent. Ship consistently.

Created by

Ansh Banga

linkedin.com/in/ansh-banga-775275330